Nyaya Saathi — API Reference

Base URL: `/api/v1` **Version:** 1.0 (Hackathon / Pilot Build) **Audience:** Frontend engineers, integrators, and technical reviewers.

---

1. Conventions

1.1 Authentication

All endpoints except `POST /auth/register`, `POST /auth/login`, `GET /health`, and `GET /ready` require authentication via a bearer token (JWT) or an authenticated session, sent as:

```
Authorization: Bearer <token>
```

1.2 Response envelope

Success:

```
{
  "data": { },
  "meta": {
    "request_id": "a1b2c3d4-..."
  }
}
```

Error:

```
{
  "error": {
    "code": "CASE_NOT_FOUND",
    "message": "Case not found.",
    "request_id": "a1b2c3d4-..."
  }
}
```

Stack traces and internal error detail are never returned to clients.

1.3 Authorization model

Every case-scoped or evidence-scoped request is checked server-side against `current_user.id == case.user_id` (or the equivalent ownership chain for evidence/documents). A client-supplied `user_id` is never trusted. Requests for a resource you don't own return `403 FORBIDDEN` (or `404 NOT_FOUND`, depending on the endpoint's disclosure policy), not the resource itself.

1.4 Common error codes

Code	HTTP status	Meaning
`UNAUTHORIZED`	401	Missing or invalid credentials
`FORBIDDEN`	403	Authenticated, but not authorized for this resource
`CASE_NOT_FOUND`	404	Case does not exist or is not visible to this user
`EVIDENCE_NOT_FOUND`	404	Evidence item does not exist or is not visible to this user
`VALIDATION_ERROR`	422	Request body failed schema validation
`FILE_TOO_LARGE`	413	Upload exceeds the configured size limit
`UNSUPPORTED_FILE_TYPE`	415	MIME type not in the allowed list
`RATE_LIMITED`	429	Too many requests in the current window
`RETRIEVAL_INSUFFICIENT`	200 (flag in payload)	RAG could not find sufficient authoritative sources; response says so instead of fabricating an answer
`INTERNAL_ERROR`	500	Unexpected server error

---

2. Auth

`POST /auth/register`

Create a new user account.

Request body

```
{
  "email": "user@example.com",
  "password": "••••••••",
  "name": "Asha Patel",
  "preferred_language": "en"
}
```

Response `201 Created`

```
{
  "data": {
    "id": "uuid",
    "email": "user@example.com",
    "name": "Asha Patel",
    "role": "USER"
  }
}
```

`POST /auth/login`

Authenticate and receive a session token.

Request body

```
{ "email": "user@example.com", "password": "••••••••" }
```

Response `200 OK`

```
{ "data": { "access_token": "jwt-token", "token_type": "bearer" } }
```

`POST /auth/logout`

Invalidate the current session/token.

`GET /auth/me`

Return the authenticated user's profile.

Response `200 OK`

```
{
  "data": {
    "id": "uuid",
    "email": "user@example.com",
    "name": "Asha Patel",
    "preferred_language": "en",
    "role": "USER"
  }
}
```

---

3. Cases

`POST /cases`

Create a new case from an initial problem description.

Request body

```
{ "title": "Unpaid salary from employer", "description": "My employer has not paid my salary for three months." }
```

Response `201 Created`

```
{
  "data": {
    "id": "uuid",
    "title": "Unpaid salary from employer",
    "status": "intake",
    "created_at": "2026-08-23T10:00:00Z"
  }
}
```

`GET /cases`

List the authenticated user's cases (paginated).

Query params: `status`, `category`, `page`, `page_size`

`GET /cases/{case_id}`

Retrieve full case detail: description, classification, urgency, jurisdiction, status, and linked evidence/action-plan/document summaries.

`PATCH /cases/{case_id}`

Update mutable case fields (e.g. title, status, answers to follow-up questions).

`DELETE /cases/{case_id}`

Delete a case and its associated evidence, subject to the platform's data retention policy.

---

4. AI Workflow

These endpoints drive the AI orchestrator for a specific case.

`POST /cases/{case_id}/classify`

Classify the case's legal category/subcategory and generate follow-up questions.

Response `200 OK`

```
{
  "data": {
    "category": "employment",
    "subcategory": "unpaid_wages",
    "urgency": "medium",
    "classification_confidence": 0.91,
    "follow_up_questions": [
      "Which state are you working in?",
      "Are you an employee or contractor?",
      "Do you have an employment agreement or salary records?"
    ]
  }
}
```

> `classification_confidence` *reflects topical classification confidence — it is never a legal-correctness score.*

`POST /cases/{case_id}/questions`

Submit answers to the follow-up questions generated above.

Request body

```
{ "answers": [ { "question_id": "uuid", "answer": "Maharashtra" } ] }
```

`POST /cases/{case_id}/analyze`

Run full RAG retrieval + grounded explanation generation for the case, incorporating any uploaded evidence.

Response `200 OK`

```
{
  "data": {
    "issue_summary": "...",
    "relevant_information": "...",
    "evidence_observed": ["..."],
    "missing_information": ["..."],
    "cautions": ["..."],
    "sources": [
      { "title": "The Payment of Wages Act", "authority": "Government of India",
"source_url": "https://www.indiacode.nic.in/...", "section_label": "Section 5" }
    ]
  }
}
```

If retrieval does not surface sufficient authoritative material, the response states that explicitly instead of fabricating a legal answer.

`POST /cases/{case_id}/action-plan`

Generate (or regenerate) the personalised, step-by-step action plan for the case.

Response `200 OK`

```
{
  "data": {
    "id": "uuid",
    "title": "Recovering unpaid wages",
    "steps": [
      { "order": 1, "title": "Send a written wage demand to your employer",
"state": "current" },
      { "order": 2, "title": "File a complaint with the labour commissioner if
unresolved in 15 days", "state": "waiting" }
    ]
  }
}
```

---

5. Evidence

`POST /cases/{case_id}/evidence`

Upload an evidence file (PDF, image, or screenshot). `multipart/form-data`.

Response `202 Accepted`

```
        { "data": { "id": "uuid", "original_filename": "bank_statement.pdf",
"processing_status": "queued" } }
```

Processing (OCR / text extraction / metadata extraction) runs asynchronously; poll `GET /evidence/{evidence_id}` or the case detail endpoint for status.

`GET /cases/{case_id}/evidence`

List all evidence attached to a case, with `processing_status` and extracted metadata summaries (not raw file contents).

`GET /evidence/{evidence_id}`

Retrieve a single evidence item's metadata and extracted summary. Actual file bytes are served only via a short-lived, signed download URL included in this response — never a public link.

`DELETE /evidence/{evidence_id}`

Delete an evidence file and its extracted data.

---

6. Documents

`POST /cases/{case_id}/documents`

Generate a document draft (e.g. a complaint letter or wage demand notice) based on the case's classification, evidence, and action plan.

Request body

```
        { "document_type": "complaint_letter" }
```

Response `201 Created`

```
        { "data": { "id": "uuid", "document_type": "complaint_letter", "version": 1 } }
```

`GET /cases/{case_id}/documents`

List all generated documents for a case.

`GET /documents/{document_id}`

Retrieve a specific document's content and download link.

---

7. Sources

`GET /cases/{case_id}/sources`

Return the full list of legal sources cited across the case's analysis, with title, authority, jurisdiction, section label, and source URL — so a user can verify every claim independently.

---

8. Health

`GET /health`

Liveness check. Returns `200 OK` if the API process is running.

`GET /ready`

Readiness check. Returns `200 OK` only once the API can reach its database and other required dependencies — used by orchestration/deployment tooling.

---

9. Rate Limiting & Abuse Controls

Authentication and AI-workflow endpoints are rate-limited per user/IP. Requests exceeding the limit receive `429 RATE_LIMITED` with a `Retry-After` header. File uploads are capped by size and validated by MIME type and extension before processing.

---

10. Versioning

The API is versioned via the URL path (`/api/v1`). Breaking changes will be introduced under a new version prefix (`/api/v2`) rather than changing the existing contract in place.